

From Memory to Skills: Evidence-Grounded Co-Evolution Governance for Long-Horizon LLM Agents

Bo Tang^{1,2}, Yang Zhang³, Guomian Zhuang¹, Wenqiang Wei¹, Gaoyang Zheng⁴,
Lindong Xie³, Yanchao Tan⁴, Feiyu Xiong¹, Qingyu Yang⁵, Edward Chung³,
Zhiyu li²

¹MemTensor, ²University of Science and Technology of China, ³Hong Kong Polytechnic University, ⁴Fuzhou University, ⁵Xi'an Jiaotong University

Abstract

Existing memory systems for long-horizon LLM agents often retrieve prior traces as passive context rather than converting them into executable capabilities. In this paper, we propose MSCE, a training-free Memory–Skill Co-Evolution framework that organizes agent experience into grounded step traces, reusable procedural policies, and declarative environmental cognition. MSCE crystallizes evidence-backed L2 policies with positive estimated gain into callable skills that retain evidence links, applicability boundaries, decision guidance, verification rules, and reliability estimates. It further introduces reflection-weighted value backfilling, which propagates sparse terminal feedback through dense local self-reflections to produce evidence-calibrated trace values for governing memory and skill evolution. Experiments on EvoAgentBench and LoCoMo demonstrate that MSCE significantly outperforms state-of-the-art skill-augmented and memory-driven agent baselines, exhibiting strong cross-domain transferability and lifelong-evolution capabilities.

Date: July 21, 2026

Correspondence: Yang Zhang at zynolo96@outlook.com

Code: <https://github.com/MemTensor/MemOS>

1 Introduction

Large Language Model (LLM) agents are increasingly deployed for long-horizon, multi-step, and user-specific tasks involving tool use, file editing, application interaction, and workflow coordination. In such settings, long-term memory is essential for overcoming limited context windows, ensuring agents can maintain coherence, personalize behavior, and reuse experience across sessions.

Existing memory mechanisms typically store either *factual memory*, such as raw interactions, user facts, and compressed contexts [3, 16, 34, 44], or *experiential memory*, such as reflections, reasoning traces, and task summaries [18, 26, 37, 41]. These memories are often reused as passive context rather than transformed into operational knowledge. For example, repeated plugin-installation traces may reveal manifest inspection, dependency resolution, registration, reload, and availability checks, yet most systems still retrieve these traces and ask the agent to reason over them again. Moreover, after repeatedly exploring the same repository, an

agent may still re-list directories to locate tests, configuration files, or build scripts, leading to excessive token consumption.

Skills offer a more executable form of reusable knowledge: instead of merely reminding the agent of past observations, a skill provides an actionable procedure that guides future actions [2]. However, turning agent trajectories into reusable skills is difficult. Raw traces contain failed attempts, blind exploration, and environment-specific artifacts, while terminal feedback is often sparse and delayed, making step-level credit assignment uncertain. Moreover, a reusable skill requires not only a successful procedure, but also triggers, applicability boundaries, verification criteria, and lifecycle maintenance. Directly distilled skills may thus overfit to raw trajectories and be invoked in inappropriate contexts.

To address these challenges, we propose MSCE, a training-free **Memory-Skill Co-Evolution** framework that promotes skills from governed memory rather than directly distilling them from noisy trajectories. MSCE organizes experience into three memory levels. *L1 trace memory* stores grounded step-level evidence. *L2 policy memory* induces recurring procedural patterns from cross-episode traces. *L3 environmental cognition memory* abstracts declarative knowledge about environmental structure and constraints. This hierarchy separates evidence, procedure, and environmental knowledge, converting noisy interaction histories into governable abstractions that can be verified, revised, and deployed as reusable skills.

On top of this governed memory, MSCE crystallizes useful and evidence-backed L2 policies into skills. A policy becomes eligible when it retains supporting evidence, exhibits positive estimated gain, and remains consistent with its current trigger, procedure, and applicability boundary. The resulting skills preserve evidence anchors, verification rules, and a reliability-driven lifecycle. To provide the value signal underlying this governance, MSCE introduces reflection-weighted value backfilling, which propagates sparse terminal feedback over grounded traces through local self-reflections. The resulting evidence-calibrated trace values serve as unified signals for retrieval, policy induction, environmental cognition abstraction, skill promotion, and later revision.

In summary, our main contributions are:

- **Framework:** We propose MSCE, a training-free memory-skill co-evolution framework that organizes long-horizon agent experience into grounded traces, procedural policies, and declarative environmental cognition.
- **Method:** We design a governed memory-to-skill promotion mechanism that crystallizes only supported, positive-gain, and stable L2 policies into callable skills. We further introduce reflection-weighted value backfilling, which couples sparse terminal feedback with dense self-reflections to produce evidence-calibrated trace values for jointly updating memory and skills.
- **Experiments:** Comprehensive evaluations on EvoAgentBench and LoCoMo show that MSCE achieves state-of-the-art performance. Further analyses validate its effectiveness in handling long-horizon and multi-domain environments, showcasing positive cross-domain transfer and lifelong-evolution gains.

2 Related Work

2.1 Memory for LLM Agents

Following Wang et al. [31], we broadly categorize existing long-term memory systems for LLM agents into two paradigms: factual memory and experiential memory. *Factual memory* focuses on retrieving raw interaction logs or observations to maintain consistency. Methods in this paradigm enhance retrieval mechanisms [3, 19] or leverage knowledge organization, structuring memory into interconnected networks [34], neurobiologically inspired graphs [6, 10], or hierarchical storage [27]. While effective for recalling past events [43, 44], they often overwhelm the context window with episodic noise and lack higher-level abstraction.

To address this, *experiential memory* summarizes past trajectories to guide future reasoning. TiM [18] enables agents to learn from trial-and-error by storing evolving chains-of-thought or reflections. Architecture-driven frameworks have introduced OS-like systems [12, 16] and advanced abstraction mechanisms [23, 31, 36, 37] to dynamically manage context and distill reusable strategies. Despite these advances, most retrieved memories

remain passive context and require the agent to reason over them again. MSCE differs by treating memory as a governed substrate from which procedural policies and deployable skills are induced.

2.2 Skills for LLM Agents

Unlike generic experience summaries, skills are structured, self-contained packages encoding reusable procedures that enable agents to execute complex workflows directly. Recent skill acquisition methods generally follow two paradigms. *External knowledge-based skills* rely on human expertise or curated datasets [4, 14, 17]. For instance, CUA-Skill parameterizes human computer-use knowledge, while SkillNet constructs large-scale skill networks. However, acquiring high-quality expertise is costly, and these skills struggle to adapt to user-specific, open-ended environments. *Interaction trajectory-based skills* enable autonomous adaptation by distilling the agent’s own trial-and-error experiences [1, 25, 28, 32, 35, 40]. For example, SkillFlow and EvoSkill introduce continuous maintenance mechanisms to extract and refine skills from execution trajectories. Besides, reinforcement learning approaches [30, 33] like SkillRL and SAGE optimize skills using environmental feedback. Despite these efforts, directly distilling skills from ungoverned raw trajectories that are inherently noisy often yields brittle, over-specific procedures. MSCE inserts an intermediate evidence-preserving memory hierarchy: L1 traces provide anchors, L2 policies aggregate cross-episode procedures, and L3 environmental cognition provides declarative context before skill deployment.

Concurrent works like MemSkill [38] and ProcMEM [22] integrate memory and skills by recasting memory operations as skills or viewing skill pools as procedural memory. In contrast, MSCE separates three-level memory and executable skills, and links them through evidence anchors and value signals. This design turns memory-skill integration into a governed promotion problem: what becomes a skill, when it applies, and how it is revised or retired. We further discuss related work on self-evolving LLM agents in Appendix A.1.

3 Background

In this paper, we formulate the LLM agent’s interaction with a long-horizon environment across an ordered sequence of tasks. A task denotes a user-facing objective initiated by a query q , while an episode denotes the feedback and update unit used by MSCE. In the common single-turn case, a task contains one episode; in multi-turn settings, a task may contain multiple episodes due to follow-up requests or corrections. We consider an ordered sequence of episodes $\mathcal{E} = \{E_1, E_2, \dots, E_n\}$. During episode E_i , the agent generates H_i step traces $F_i = \{f_{i,1}, f_{i,2}, \dots, f_{i,H_i}\}$. At each step t , the trace is represented as $(s_{i,t}, a_{i,t}, o_{i,t}, \rho_{i,t})$, where $s_{i,t}$ denotes the semantic context, $a_{i,t}$ is the primitive action, $o_{i,t}$ is the observation returned by the environment, and $\rho_{i,t}$ is the agent’s self-reflection.

Within this environment, the agent can leverage a crystallized skill library $\mathcal{K}$. Rather than exploring from scratch at every step, the agent monitors whether the current state $s_{i,t}$ satisfies the trigger condition of any skill in $\mathcal{K}$. If a relevant skill $k = \mu(s_{i,t}, \mathcal{K})$ is retrieved, it is injected into the context to guide action generation: $a_{i,t} = \pi_{\text{LLM}}(\cdot | s_{i,t}, k)$. Otherwise, the policy falls back to generating actions conditioned only on the current state. After episode E_i terminates, the system receives a terminal feedback signal. This signal may be a numerical reward from an environment verifier or textual feedback from the user. For textual feedback, we use the reward quantification mechanism in Appendix B.3 to convert it into a scalar terminal feedback R_i . The primary objective is to maximize cumulative episode-level feedback $\sum_{i=1}^n R_i$. In practice, R_i may reflect task success, user satisfaction, process quality, safety, or efficiency.

4 Method

4.1 Overview

The overview structure of MSCE is illustrated in Fig. 1. It maintains a memory hierarchy $\mathcal{M} = (\mathcal{M}^{(1)}, \mathcal{M}^{(2)}, \mathcal{M}^{(3)})$, where L1 traces preserve auditable evidence, L2 policies abstract revisable procedures, and L3 environmental cognition stores declarative environment knowledge. The skill library $\mathcal{K}$ is downstream of this memory hierarchy, exposing verified procedures for invocation. This separation keeps skills traceable to evidence while preventing traces, policies, environmental facts, and callable actions from being merged into a single uncon-

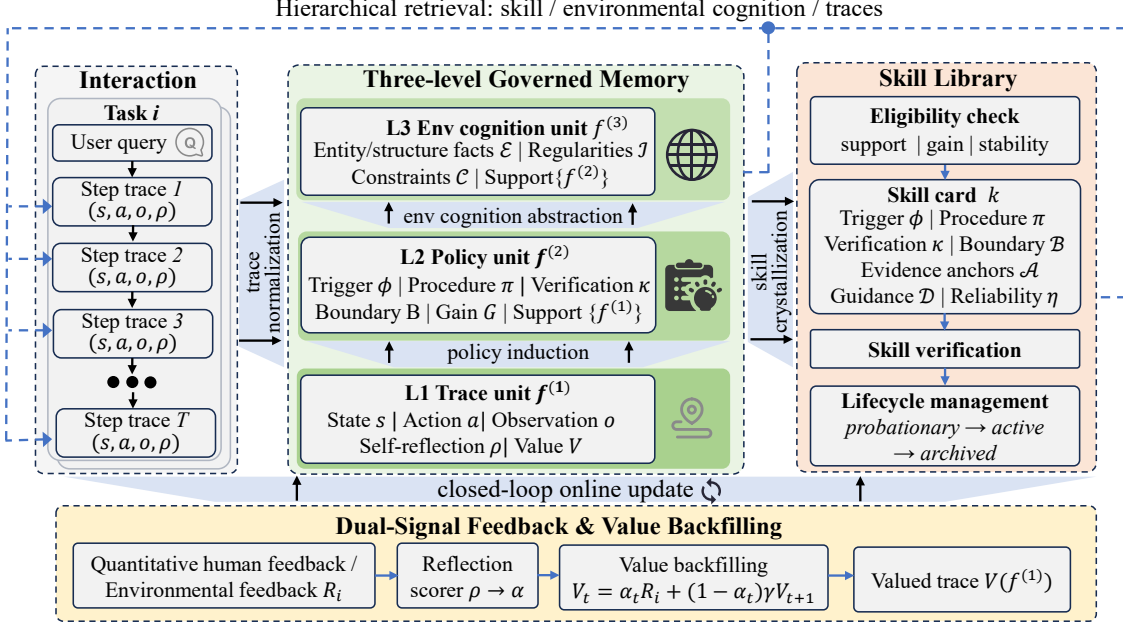


Figure 1 Overview of MSCE with governed memory, skill crystallization, and dual-signal value backfilling.

trolled pool. MSCE crystallizes evidence-backed L2 policies with positive estimated gain into skills, and uses reflection-weighted value backfilling to update memory and skills from terminal feedback.

4.2 Three-level Memory

L1 Trace Memory. L1 stores grounded decision units collected during interaction:

$$f_{i,t}^{(1)} = (s_{i,t}, a_{i,t}, o_{i,t}, \rho_{i,t}, V(f_{i,t}^{(1)})),$$

where $s_{i,t}$, $a_{i,t}$, $o_{i,t}$, and $\rho_{i,t}$ denote the semantic state, action, observation, and self-reflection at step t . The trace value $V(f_{i,t}^{(1)})$ is initialized as unavailable and filled after terminal feedback arrives, as described in Section 4.4.

L1 is the evidence layer of MSCE: higher-level memories and skills must maintain links to their supporting traces, so that induced procedures can be traced back to concrete interactions.

To bound storage and reduce privacy risk, MSCE does not persist unbounded raw observations. Instead, L1 stores normalized evidence: truncated state text, agent action text, structured tool-call metadata, and capped tool inputs and outputs. Higher-level memories store evidence identifiers rather than duplicating raw observations.

L2 Policy Memory. L2 stores reusable policies induced from recurring trace evidence:

$$f^{(2)} = (\phi, \pi, \kappa, \mathcal{B}, \{f^{(1)}\}),$$

where ϕ is the trigger condition, π is a natural-language procedure, κ specifies verification or fallback criteria, $\mathcal{B}$ records applicability boundaries, and $\{f^{(1)}\}$ is the supporting L1 evidence set.

Policy extraction. When a valued L1 trace $f_{i,t}^{(1)}$ becomes available, MSCE first checks whether its step value satisfies $V(f_{i,t}^{(1)}) \geq v_{\min}$ where $v_{\min}$ denotes the threshold for policy extraction. Eligible traces are matched against existing L2 policies using a combination of embedding similarity and structured evidence such as domain tags, tool types, and normalized error signatures. A matched trace updates the corresponding policy. If no policy matches, the trace is placed into a candidate pool keyed by a deterministic pattern signature. A

new L2 policy is induced by the prompt Π_{policy} only when the same signature bucket contains evidence from at least n_{min} distinct episodes.

Policy gain. For each touched policy, MSCE computes a heuristic utility gain over per-step values. Let S_{with} be traces linked to the policy and S_{without} be other traces in the same evaluation pool. The gain is

$$G(f^{(2)}) = \bar{V}_{\text{with}} - \bar{V}_{\text{blend}}(S_{\text{without}}). \quad (1)$$

When $|S_{\text{with}}| \geq 3$, $\bar{V}_{\text{with}}$ is a softmax-weighted mean over $V(f)$; else it falls back to the arithmetic mean. The without-side aggregate is an arithmetic mean shrunk toward a baseline. Appendix B.4 gives the full definition. We use G as a heuristic signal rather than a causal effect estimate: it determines whether a policy remains active, becomes eligible for skill crystallization, or is retired.

L3 Environmental Cognition Memory. L3 compresses environment-level cognition as

$$f^{(3)} = (\mathcal{E}, \mathcal{I}, \mathcal{C}, \{f^{(2)}\}),$$

where $\mathcal{E}$, $\mathcal{I}$, and $\mathcal{C}$ are entity or structure facts, action–response regularities, and environmental constraints, each linked to supporting L2 policies $\{f^{(2)}\}$. Unlike L2 policies outlining how to solve recurring tasks, L3 describes how a particular environment or domain is organized. For example, repeated policies about editing frontend files, locating tests, and modifying configuration files may support a repository environmental cognition.

L3 abstraction is triggered by prompt Π_{env} when multiple active policies share a domain or environment context. The abstraction step extracts declarative facts from the contributing policies and their evidence, while excluding imperative procedural instructions. When a new L3 draft overlaps with an existing cognition, MSCE merges it; otherwise, it creates a new environmental cognition. When later evidence weakens support for a cognition, its confidence and retrieval priority can be reduced; low-confidence cognitions are retained for audit but excluded from default retrieval.

4.3 Skill Crystallization and Invocation

A skill is a callable object derived from an eligible L2 policy:

$$k = (\phi, \pi, \kappa, \mathcal{B}, \mathcal{A}, \mathcal{D}, \eta).$$

It inherits the trigger, procedure, verification rule, and boundary from L2, adding evidence anchors $\mathcal{A}$, decision guidance $\mathcal{D}$, and reliability η . While L2 policies and skills share several fields, they serve different roles. An L2 policy is an internal abstraction for evidence aggregation and revision; it may be incomplete, unstable, or not directly exposed to the agent. A skill is a deployment object with a standardized invocation interface, lifecycle state, reliability estimate, evidence anchors, and decision guidance. Thus, crystallization is not merely formatting an L2 policy, but a promotion step that makes a policy callable only after evidence, gain, and recent consistency checks.

Skill promotion is governed by two gates. First, the source L2 policy must retain supporting traces and satisfy the positive-gain condition $G(f^{(2)}) > \theta_G$ where θ_G is a threshold for skill promotion. Second, it must be stable: recent evidence should fit the current trigger, procedure, and boundary, rather than forcing a substantial rewrite of ϕ , π , or $\mathcal{B}$. When these gates are satisfied, MSCE constructs a support set from positive evidence and counter-evidence. Positive evidence induces the common procedure, while counter-evidence constrains the boundary and yields anti-patterns.

Crystallization and verification. The crystallization prompt Π_{skill} conditions on the policy, evidence anchors $\mathcal{A}$, a whitelist of tools observed in evidence, and any decision-guidance seeds already attached to the policy. It returns structured steps, parameters, and an initial decision guidance $\mathcal{D} = (\text{preferences}, \text{anti-patterns})$. A deterministic verifier checks each draft skill before insertion. It validates the required schema, including the skill name, preconditions, ordered steps, examples, tool list, and decision-guidance fields. It then enforces evidence grounding: cited evidence identifiers must come from the support set, and declared tools from the

whitelist observed in evidence traces. Finally, two lightweight coverage tests ensure that the draft neither invents unsupported commands nor deviates from the retained trace evidence. Drafts that fail any check are discarded rather than exposed as callable skills.

Deployment and reliability. After deployment, MSCE estimates skill reliability with a smoothed success rate. Let n_{pass} and n_{trial} denote successful and total invocations, respectively:

$$\eta = \frac{n_{\text{pass}} + 1}{n_{\text{trial}} + 2}.$$

User correction or rejection decreases η and may shrink $\mathcal{B}$, while repeated successes increase it. High-reliability skills enter active retrieval; low-reliability skills remain probationary or are archived. During inference, MSCE retrieves active skills first, but conditions their invocation on the memory hierarchy. If the context also matches an L3 cognition, retrieved L3 facts provide environmental priors that instantiate parameters and interpret applicability boundaries without overwriting the skill procedure. For example, a dependency-installation skill may choose package-manager commands according to whether the L3 cognition indicates Alpine, Debian, or macOS. If no skill applies or a skill fails, MSCE falls back to L1 traces and episode-level retrieval.

4.4 Online Update via Dual-Signal Value Backfilling

MSCE does not update the base LLM. Instead, it updates the external cognitive state

$$\mathcal{C}_i = (\mathcal{M}^{(1)}, \mathcal{M}^{(2)}, \mathcal{M}^{(3)}, \mathcal{K})_i$$

through two complementary feedback sources: dense step-level reflections and sparse terminal feedback. Reflections are available at most interaction steps but may be noisy or unfaithful; terminal feedback is more reliable but delayed and sparse. MSCE couples these signals by reflection-weighted value backfilling.

To distribute the sparse terminal feedback R_i to each step trace, the terminal step inherits this value: $V(f_{i,H_i}^{(1)}) = R_i$. For earlier steps, MSCE applies reflection-weighted value backfilling:

$$V(f_{i,t}^{(1)}) = \alpha_{i,t} R_i + (1 - \alpha_{i,t}) \gamma V(f_{i,t+1}^{(1)}), \quad (2)$$

where $\gamma \in [0, 1)$ is a discount factor and $\alpha_{i,t} \in [0, 1]$ is a reflection weight. Intuitively, high- α steps provide locally informative evidence for the final outcome, while low- α steps mainly inherit value from later steps. This biases memory construction toward steps that are both globally rewarded and locally interpretable. The weight α is estimated via the prompt $\Pi_{\text{reflection_score}}$ based on the self-reflection $\rho_{i,t}$ and its local context: faithful, concrete, causally informative, and transferable reflections receive higher weights, whereas empty, tautological, or unsupported ones receive zero or low weights.

Online update. After an episode closes, $\mathcal{U}$ executes asynchronously. It first persists normalized L1 traces and embeddings. Once terminal feedback R_i is available, MSCE backfills step values V . The valued traces are then used to associate or induce L2 policies; active policies may trigger L3 abstraction, and eligible policies may crystallize into skills. Tool-failure bursts or explicit user corrections extend the decision-guidance field $\mathcal{D}$, and the resulting repair context is injected on the next turn rather than interrupting the current action.

Additional details. Appendix C provides a concrete example illustrating the MSCE workflow. Appendix B gives implementation details on agent reflection, skill lifecycle, memory retrieval, decision guidance, and storage and privacy controls. Appendix E lists the prompts used by MSCE.

5 Experiments

Table 1 Main results on EvoAgentBench. We report Pass@1 (%) and Cost across five domains. Cost is measured in **chars** for Math and **turns** for other domains. For Pass@1, the best results are shown in bold, and the second best results are underlined. Cost is reported as an efficiency reference and should be interpreted jointly with Pass@1, as lower cost may reflect premature termination or less extensive tool use.

Method	Information Retrieval		Mathematical Reasoning		Software Engineering		Code Implementation		Knowledge Work	
	Pass@1 $\uparrow$	Cost $\downarrow$	Pass@1 $\uparrow$	Cost $\downarrow$	Pass@1 $\uparrow$	Cost $\downarrow$	Pass@1 $\uparrow$	Cost $\downarrow$	Pass@1 $\uparrow$	Cost $\downarrow$
Vanilla Agent	12.31	6.9	30.00	1290.0	<u>38.46</u>	41.2	51.28	3.0	44.83	13.3
Memento	13.85	9.5	32.00	1091.3	<u>38.46</u>	37.7	<u>58.97</u>	4.1	46.55	16.4
EverOS	<u>21.54</u>	14.0	<u>43.00</u>	1745.3	<u>26.92</u>	59.7	<u>48.72</u>	7.2	<u>48.28</u>	19.1
EvoSkill	12.31	9.7	29.00	1185.9	26.92	37.9	61.54	3.9	<u>48.28</u>	15.3
OpenSpace	15.38	8.1	28.00	1468.3	30.77	48.4	56.41	4.5	31.03	13.2
MemSkill	15.38	8.4	29.00	1103.4	<u>38.46</u>	37.3	56.41	3.9	46.55	17.1
SkillFlow-Evolve	16.92	8.7	33.00	1304.0	<u>34.62</u>	56.4	56.41	4.1	46.55	11.9
MSCE	26.15	8.7	47.00	1140.9	53.85	40.8	61.54	2.0	53.45	15.0

5.1 Experimental Settings

Benchmarks. Our primary evaluation uses EvoAgentBench,¹ a multi-domain benchmark for long-horizon agent self-evolution. It covers five agent domains: information retrieval based on BrowseComp-Plus [5], mathematical reasoning based on OmniMath [7], software engineering based on SWE-Bench [15], code implementation based on LiveCodeBench [13], and knowledge work based on GDPVal [24]. We abbreviate them as IR, Math, SE, Code, and KW, respectively. We additionally evaluate on LoCoMo [21] to test long-term dialogue memory; we report it separately because it stresses memory consistency rather than tool-use skill execution.

Baselines. We compare MSCE with three categories of baselines. First, *memory-based agents*, including EverOS [12], Memento, and MemSkill, reuse past trajectories or stored cases but do not explicitly crystallize reusable skills. Second, *trajectory-to-skill agents*, including EvoSkill, OpenSpace [11], and SkillFlow-Evolve, synthesize or refine skills from interaction trajectories. SkillFlow-Evolve is a patch-based skill-evolution baseline adapted from SkillFlow [40]. Finally, we include a *vanilla agent* that uses the same base model and tools but does not persist long-term memory or construct skills.

Implementation. All methods are implemented on the OpenClaw v2026.5.7 runtime² and receive identical tool access, interaction budgets, and task order. We use GPT-5.2 as the agent backbone and GPT-4o for auxiliary prompted operators, including reflection scoring, L2 policy induction, and L3 environmental cognition abstraction. For online methods, memory and skills are accumulated only from online evolution episodes without updating the base LLM, and are then reused on later evaluation. Hyperparameters are reported in Appendix (Table 6).

Metrics. We report **Pass@1** as the main effectiveness metric for EvoAgentBench. Pass@1 is the macro-averaged per-task success rate: each trial receives a domain-specific reward r , and is counted as successful if $r > \tau_{eval}$ (default $\tau_{eval}=0$; $\tau_{eval}=0.6$ for KW). We also report **cost**, which is measured by the mean number of interaction **turns** or character count (abbreviated as **chars**) per trial; see Appendix D.1 for full evaluation protocol. On LoCoMo, we report GPT-4o judge scores for four question types, together with overall judge score and F1.

5.2 Main Results

Results on EvoAgentBench. As shown in Table 1, MSCE achieves the best or tied-best Pass@1 in all five domains. Compared with the strongest non-MSCE baseline in each domain, it improves Pass@1 by 4.61 points on IR, 4.00 on Math, 15.39 on SE, and 5.17 on KW. On Code, MSCE ties the best baseline, EvoSkill, at 61.54% Pass@1 while reducing cost from 3.9 to 2.0 turns. These results indicate that MSCE improves self-evolution across heterogeneous agent tasks.

¹<https://evermind-ai.github.io/EvoAgentBench>

²<https://openclaw.ai/>

Table 2 Results on LoCoMo. We report LLM-judge (GPT-4o) scores for four question types, including single-hop, multi-hop, temporal reasoning, and open-domain questions, together with the overall judge score and overall F1 score. Higher is better for all metrics.

Method	Single	Multi	Temp.	Open	Overall	F1
Vanilla Agent	28.18	21.63	12.15	39.58	24.35	25.95
Memento	48.51	14.18	28.04	13.54	35.78	30.02
EverOS	59.93	46.10	38.32	<u>29.17</u>	50.97	41.58
EvoSkill	42.45	27.30	19.94	16.67	33.38	25.51
OpenSpace	54.93	37.59	34.89	21.88	45.52	36.94
MemSkill	52.91	29.43	31.46	13.54	41.69	34.00
SkillFlow-Evolve	<u>74.08</u>	<u>46.81</u>	<u>41.43</u>	25.00	<u>59.22</u>	<u>48.71</u>
MSCE	75.98	47.87	44.24	28.13	61.23	49.89

Table 3 Ablation results on EvoAgentBench. The top row shows the full MSCE method, while the following rows evaluate non-hierarchical and component-level ablations.

Variant	Information Retrieval		Mathematical Reasoning		Software Engineering		Code Implementation		Knowledge Work	
	Pass@1 ↑	Cost ↓	Pass@1 ↑	Cost ↓	Pass@1 ↑	Cost ↓	Pass@1 ↑	Cost ↓	Pass@1 ↑	Cost ↓
Full MSCE	26.15	8.7	47.00	1140.9	53.85	40.8	61.54	2.0	53.45	15.0
Flat Memory	10.77	9.0	31.00	1322.2	34.62	56.0	56.41	5.3	37.93	17.7
w/o L3	23.08	11.1	43.00	1461.3	50.00	54.6	58.97	2.8	48.28	19.5
w/o Value Calibration	24.62	9.7	44.00	1297.3	50.00	46.4	58.97	2.3	50.00	16.4
w/o Reflection Weighting	21.54	10.8	40.00	1378.2	46.15	50.7	56.41	2.5	46.55	17.6
w/o Skill Crystallization	20.00	12.6	39.00	1568.6	42.31	58.8	53.85	3.1	44.83	20.3

The cost results show that these gains are not obtained by simply spending more inference budget. When comparing against the strongest non-MSCE baseline by Pass@1, breaking ties by lower cost, MSCE reduces cost on IR, Math, Code, and KW, with especially large reductions on Math and Code. The only exception is SE, where MSCE improves Pass@1 by 15.39 points but increases cost from 37.3 to 40.8 turns, consistent with the additional repository navigation, editing, and testing often required by successful software-engineering solutions. Overall, MSCE achieves the strongest accuracy while maintaining favorable efficiency in most settings.

Long-dialogue memory results. Table 2 reports LoCoMo results. MSCE achieves the best overall judge score and F1, outperforming the strongest baseline, SkillFlow-Evolve, by 2.01 and 1.18 points, respectively. It also obtains the highest scores on single-hop, multi-hop, and temporal reasoning questions, indicating that the governed L1–L2–L3 hierarchy helps preserve factual evidence while abstracting long-range dependencies. EverOS performs best on open-domain questions excluding Vanilla Agent, but MSCE ranks second on this category and achieves the strongest aggregate performance.

Simulated human feedback. Appendix D.2 further studies MSCE with LLM-simulated human feedback injected at the episode level. The results show that the introduction of simulated human feedback improves Pass@1 on most EvoAgentBench domains, suggesting that richer textual feedback can provide useful learning signals for memory–skill evolution.

5.3 Generalization Analysis

Cross-domain transfer. We evaluate whether evolved memory and skills transfer across domains: for each pair $A \rightarrow B$, *intra* evolves and evaluates the agent on target domain B , whereas *cross* initializes it with memory and skills evolved on source domain A before further evolution and evaluation on B .

Figure 2 shows that cross-domain transfer improves Pass@1 across all six transfer pairs, with gains from +2.56 to +5.13 points and an average gain of +3.93 points, with cost decreasing in four of six pairs. The two transfers into Code increase cost by +4.6% and +8.1%, but also yield clear Pass@1 gains, suggesting

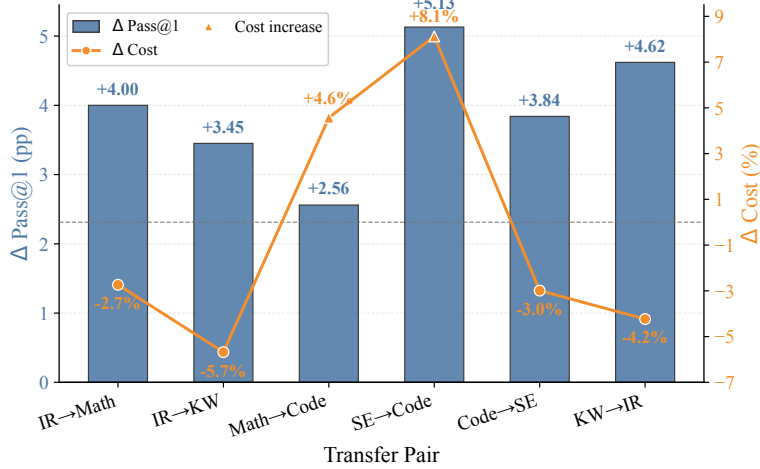


Figure 2 Cross-domain transfer results. Bars show the absolute Pass@1 improvement from *intra* to *cross* in percentage points, and the orange line shows the relative cost change.

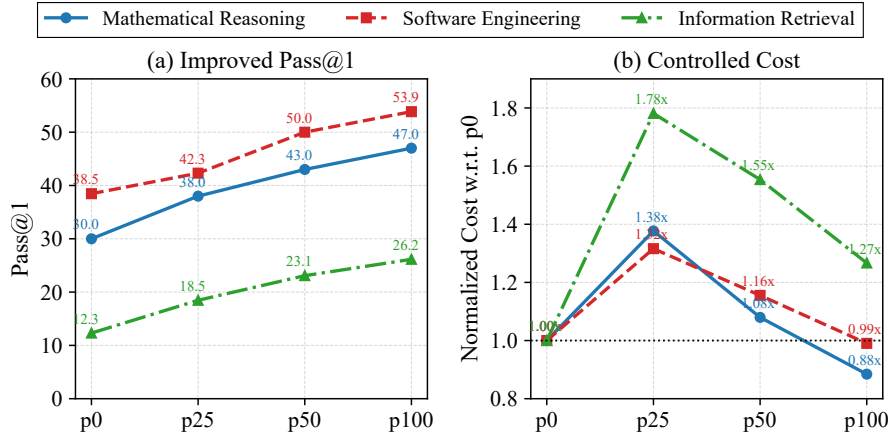


Figure 3 Long-horizon cumulative evolution under the lifelong learning protocol. The left panel shows Pass@1 across increasing accumulated training scales, and the right panel shows cost normalized by the corresponding p_0 cost of each task category.

a favorable accuracy–cost trade-off. These results indicate that MSCE learns transferable problem-solving structure rather than domain-specific artifacts.

Lifelong Evolution Figure 3 presents long-horizon evolution under the lifelong learning protocol, where p_0 denotes the initial no-accumulation setting and p_{25} , p_{50} , and p_{100} denote increasing accumulated experience scales. Pass@1 improves monotonically from p_0 to p_{100} , with gains of 17.00, 15.39, and 13.84 points on Math, SE, and IR, respectively. Normalized cost first increases at p_{25} and then consistently decreases toward p_{100} . At p_{100} , cost becomes lower than p_0 on Math and SE and is substantially reduced from its p_{25} peak on IR. These results support a “learning by using” effect: accumulated experience improves success without simply increasing inference cost.

5.4 Ablation Study

We ablate five components of MSCE. **Flat Memory** stores all experiences in a single non-hierarchical memory and retrieves them directly at test time, without hierarchical abstraction, applicability gating, or skill introduction. **w/o L3** removes L3 environmental cognition abstraction. **w/o Value Calibration** directly injects retrieved skills without value-calibrated applicability filtering. **w/o Reflection Weighting** removes reflection-weighted value backfilling and treats feedback uniformly. **w/o Skill Crystallization** prevents distilled task policies from being converted into reusable skills.

Table 3 shows that all components contribute to performance and efficiency. The largest degradation comes from replacing MSCE with **Flat Memory**: Pass@1 drops by 15.38, 16.00, and 19.23 points on IR, Math, and SE, respectively, and Code cost increases from 2.0 to 5.3 turns despite a relatively small Pass@1 drop. This indicates that merely retrieving past experiences is insufficient; effective reuse requires hierarchical abstraction, applicability control, and skill-level consolidation.

Among component-level ablations, removing skill crystallization causes the most consistent degradation, reducing Pass@1 by 6.15–11.54 points across domains and increasing cost for every domain. Removing the L3 environmental cognition, reflection weighting, or the value calibration also consistently hurts performance. In particular, disabling the value calibration lowers Pass@1 and increases cost in every domain, supporting the need for applicability-aware filtering to avoid harmful skill injection. Overall, the ablations show that MSCE’s gains arise from the joint effect of hierarchical organization, skill crystallization, calibrated selection, and reflection-weighted learning.

6 Conclusion

In this paper, we introduced MSCE, a training-free memory–skill co-evolution framework for long-horizon LLM agents. By organizing experience into grounded traces, reusable policies, and environmental cognition, and by crystallizing evidence-backed, positive-gain policies into skills, MSCE bridges the gap between passive memory retrieval and active skill execution. Extensive experiments validate that MSCE achieves state-of-the-art performance on long-horizon self-evolution and conversational memory benchmarks, alongside significant cross-domain transfer and lifelong-evolution capabilities. We hope MSCE provides a foundation for connecting memory and skills, contributing to the development of more efficient and autonomously evolving LLM agents.

Limitations

MSCE has several limitations. First, its value estimates, policy gains, and reliability scores are heuristic governance signals rather than causal credit-assignment estimates. They support ranking, filtering, and revision of memory objects, but do not guarantee that a promoted policy would cause success under counterfactual intervention.

Second, several update operators, including reflection scoring, policy induction, environmental cognition abstraction, reward quantification, and skill drafting, rely on prompted LLMs. Although outputs are evidence-constrained and verified before insertion, they may still be noisy or sensitive to model and prompt choices, and they introduce additional latency and cost.

Third, our implementation depends on a particular agent runtime, tool interface, and backbone configuration; absolute results may vary under different models or environments.

Finally, MSCE stores normalized traces, evidence identifiers, policies, environmental cognition, and skills. Although we apply truncation, deduplication, and rule-based redaction to reduce storage and privacy risks, these controls do not eliminate the possibility of retaining sensitive user information, environment-specific secrets, or unsafe procedural knowledge. Practical deployments should include stronger privacy filters, access control, and safety checks for generated skills.

References

- [1] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. arXiv preprint arXiv:2603.02766, 2026.
- [2] Anthropic. Equipping agents for the real world with agent skills, October 2025. URL <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>. Engineering at Anthropic.
- [3] Nuo Chen, Hongguang Li, Jianhui Chang, Juhua Huang, Baoyuan Wang, and Jia Li. Compress to impress: Unleashing the potential of compressive memory in real-world long-term conversations. In Proceedings of the 31st International Conference on Computational Linguistics, pages 755–773, 2025.
- [4] Tianyi Chen, Yinheng Li, Michael Solodko, Sen Wang, Nan Jiang, Tingyuan Cui, Junheng Hao, Jongwoo Ko, Sara Abdali, Leon Xu, et al. Cua-skill: Develop skills for computer using agent. arXiv preprint arXiv:2601.21123, 2026.
- [5] Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, et al. Browsecomp-plus: A more fair and transparent evaluation benchmark of deep-research agent. arXiv preprint arXiv:2508.06600, 2025.
- [6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. arXiv preprint arXiv:2504.19413, 2025.
- [7] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Zhengyang Tang, et al. Omni-math: A universal olympiad level mathematic benchmark for large language models. In International Conference on Learning Representations, volume 2025, pages 100540–100569, 2025.
- [8] Rui Ge, Yichao Fu, Yuyang Qian, Junda Su, Yiming Zhao, Peng Zhao, and Hao Zhang. Internalizing agency from reflective experience. arXiv preprint arXiv:2603.16843, 2026.
- [9] Yubin Ge, Salvatore Romeo, Jason Cai, Monica Sunkara, and Yi Zhang. Samule: Self-learning agents enhanced by multi-level reflection. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 16602–16621, 2025.
- [10] Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. Advances in neural information processing systems, 37:59532–59569, 2024.
- [11] HKUDS. OpenSpace: Make Your Agents: Smarter, Low-Cost, Self-Evolving. <https://github.com/HKUDS/OpenSpace>, 2026. Accessed: 2026-05-22.
- [12] Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, et al. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning. arXiv preprint arXiv:2601.02163, 2026.
- [13] Naman Jain, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In International Conference on Learning Representations, volume 2025, pages 58791–58831, 2025.
- [14] Zhengbo Jiao, Shaobo Wang, Zifan Zhang, Xuan Ren, Wei Wang, Bing Zhao, Hu Wei, and Linfeng Zhang. Agentic proposing: Enhancing large language model reasoning via compositional skill synthesis. arXiv preprint arXiv:2602.03279, 2026.
- [15] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In International Conference on Learning Representations, volume 2024, pages 54107–54157, 2024.
- [16] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory os of ai agent. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 25972–25981, 2025.
- [17] Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, et al. Skillnet: Create, evaluate, and connect ai skills. arXiv preprint arXiv:2603.04448, 2026.
- [18] Lei Liu, Xiaoyan Yang, Yue Shen, Binbin Hu, Zhiqiang Zhang, Jinjie Gu, and Guannan Zhang. Think-in-memory: Recalling and post-thinking enable llms with long-term memory. arXiv preprint arXiv:2311.08719, 2023.

- [19] Lin Long, Yichen He, Wentao Ye, Yiyuan Pan, Yuan Lin, Hang Li, Junbo Zhao, and Wei Li. Seeing, listening, remembering, and reasoning: A multimodal agent with long-term memory. [arXiv preprint arXiv:2508.09736](#), 2025.
- [20] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. [Advances in neural information processing systems](#), 36:46534–46594, 2023.
- [21] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In [Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics \(Volume 1: Long Papers\)](#), pages 13851–13870, 2024.
- [22] Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Procmem: Learning reusable procedural memory from experience via non-parametric ppo for llm agents. [arXiv preprint arXiv:2602.01869](#), 2026.
- [23] Siru Ouyang, Jun Yan, I Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T Le, Samira Daruki, Xiangru Tang, et al. Reasoningbank: Scaling agent self-evolving with reasoning memory. [arXiv preprint arXiv:2509.25140](#), 2025.
- [24] Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubei, Phoebe Thacker, Laurance Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. [arXiv preprint arXiv:2510.04374](#), 2025.
- [25] Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: From trajectories to reusable expertise for continual llm agent refinement. [arXiv preprint arXiv:2601.22758](#), 2026.
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. [Advances in neural information processing systems](#), 36:8634–8652, 2023.
- [27] Haoran Sun, Shaoning Zeng, and Bob Zhang. H-mem: Hierarchical memory for high-efficiency long-term reasoning in llm agents. In [Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics \(Volume 1: Long Papers\)](#), pages 341–350, 2026.
- [28] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. [Transactions on Machine Learning Research](#), 2023.
- [29] Hanyu Wang, Yuanpu Cao, Lu Lin, and Jinghui Chen. Preflect: From retrospective to prospective reflection in large language model agents. [arXiv preprint arXiv:2602.07187](#), 2026.
- [30] Jiong Xiao Wang, Qiaojing Yan, Yawei Wang, Yijun Tian, Soumya Smruti Mishra, Zhichao Xu, Megha Gandhi, Panpan Xu, and Lin Lee Cheong. Reinforcement learning for self-improving agent with skill library. [arXiv preprint arXiv:2512.17102](#), 2025.
- [31] Zhenting Wang, Huancheng Chen, Jiayun Wang, and Wei Wei. Memex (rl): Scaling long-horizon llm agents via indexed experience memory. [arXiv preprint arXiv:2603.04257](#), 2026.
- [32] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. In [International Conference on Machine Learning](#), pages 63897–63911. PMLR, 2025.
- [33] Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, et al. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. [arXiv preprint arXiv:2602.08234](#), 2026.
- [34] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. [arXiv preprint arXiv:2502.12110](#), 2025.
- [35] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. [arXiv preprint arXiv:2603.01145](#), 2026.

- [36] Guibin Zhang, Muxin Fu, Kun Wang, Guancheng Wan, Miao Yu, and Shuicheng YAN. G-memory: Tracing hierarchical memory for multi-agent systems. In The Thirty-ninth Annual Conference on Neural Information Processing Systems.
- [37] Guibin Zhang, Muxin Fu, and Shuicheng Yan. Memgen: Weaving generative latent memory for self-evolving agents. arXiv preprint arXiv:2509.24704, 2025.
- [38] Haozhen Zhang, Quanyu Long, Jianzhu Bao, Tao Feng, Weizhi Zhang, Haodong Yue, and Wenya Wang. Memskill: Learning and evolving memory skills for self-evolving agents. arXiv preprint arXiv:2602.02474, 2026.
- [39] Xiaoying Zhang, Zichen Liu, Yipeng Zhang, Xia Hu, and Wenqi Shao. Retroagent: From solving to evolving via retrospective dual intrinsic feedback. arXiv preprint arXiv:2603.08561, 2026.
- [40] Ziao Zhang, Kou Shi, Shiting Huang, Avery Nie, Yu Zeng, Yiming Zhao, Zhen Fang, Qishen Su, Haibo Qiu, Wei Yang, et al. Skillflow: Benchmarking lifelong skill discovery and evolution for autonomous agents. arXiv preprint arXiv:2604.17308, 2026.
- [41] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 38, pages 19632–19642, 2024.
- [42] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In Proceedings of the 41st International Conference on Machine Learning, pages 62138–62160, 2024.
- [43] Huichi Zhou, Yihang Chen, Siyuan Guo, Xue Yan, Kin Hei Lee, Zihan Wang, Ka Yiu Lee, Guchun Zhang, Kun Shao, Linyi Yang, et al. Memento: Fine-tuning llm agents without fine-tuning llms. arXiv preprint arXiv:2508.16153, 2025.
- [44] Wangchunshu Zhou, Yuchen Eleanor Jiang, Peng Cui, Tiannan Wang, Zhenxin Xiao, Yifan Hou, Ryan Cotterell, and Mrinmaya Sachan. Recurrentgpt: Interactive generation of (arbitrarily) long text. arXiv preprint arXiv:2305.13304, 2023.

Appendix

A More Related Work

A.1 Self-Evolution of LLM Agents

Self-evolving LLM agents improve future behavior from past interactions through dual-signal feedback: *internal reflection* and *external task feedback*. Reflection-based methods, such as Self-Refine [20], Reflexion [26], ExpeL [41], SAMULE [9], and PreFlect [29], generate natural-language critiques over outputs, trajectories, or plans to guide future attempts. While dense and interpretable, such reflections can be generic, mislocalized, or hallucinated. In contrast, feedback-driven methods such as LATS [42], LEAFE [8], and RetroAgent [39] exploit environment signals, verifiers, or rewards to guide search, distillation, or policy learning. These signals are more reliable but often sparse and delayed, making credit assignment difficult.

MSCE explicitly couples these two feedback sources through reflection-weighted value backfilling. This produces unified trace-level values that guide memory retrieval, policy induction, and skill crystallization, enabling memory and skills to co-evolve in a training-free manner.

B Implementation Details of MSCE

This appendix provides implementation details omitted from the main text.

B.1 Agent Reflection

The reflection field $\rho_{i,t}$ is obtained from the agent runtime when native reflective metadata is available; otherwise MSCE extracts inline reflective statements from the assistant turn, and may synthesize a short reflection with an LLM when no reflection is present. The reliability of $\rho_{i,t}$ is not assumed, but later estimated through the reflection scoring procedure in Appendix E.1.

B.2 Episode Definition and Boundary Management

The main text defines an *episode* as the feedback and update unit used by MSCE. This appendix provides the boundary-management details needed to instantiate that definition in online multi-turn interaction. An episode is not an additional memory level; rather, it determines when L1 traces are closed, when terminal feedback is backfilled, and when downstream memory updates are triggered.

Task, episode, and step. A *task* is the user-facing objective introduced in Section 3, an *episode* is the feedback and update unit, and a *step* is the minimal L1 memory unit $f_{i,t}^{(1)}$. Each episode closes a set of L1 steps that are assigned terminal feedback together. In the common single-turn case, a task and an episode coincide. In multi-turn settings, a task may contain multiple episodes when the user provides follow-up requests or corrections. MSCE uses episode boundaries to decide whether a new interaction should revise the previous evidence, continue the same session, or start a new task context.

Formally, we write an episode as

$$E_i = \{\tau_{i,1}, \tau_{i,2}, \dots, \tau_{i,H_i}\},$$

where each $\tau_{i,t}$ is an assistant response segment or a tool-mediated action–observation segment. The capture module converts E_j into an ordered set of L1 traces

$$F_i^{(1)} = \{f_{i,1}^{(1)}, f_{i,2}^{(1)}, \dots, f_{i,H_i}^{(1)}\}.$$

All traces in $F_i^{(1)}$ share the same episode identifier, which is used for reward backfilling, L2 evidence grouping, and episode-level retrieval.

Episode boundaries. MSCE maintains sessions across multiple user turns. When a new user message arrives, a lightweight relation classifier determines how it relates to the previous episode. The classifier combines lexical cues with an LLM-based tie-breaker when necessary and returns one of three operational labels:

- **Correction:** the new message corrects or rejects the previous result. MSCE extends the last episode so that the new evidence can revise the same credit-assignment unit.
- **Follow-up:** the new message extends the same goal or environment without invalidating the previous result. MSCE closes the previous episode and opens a new episode under the same task context.
- **New task:** the new message introduces an unrelated goal or domain. MSCE starts a new task context while keeping long-term memory retrievable.

This design separates the user-facing notion of a task from the system-level unit used for online updates. It also avoids treating every tool call as an independent task, which would make credit assignment too fine-grained and unstable.

Role in value backfilling. Episode boundaries define the scope of credit assignment. Once feedback for an episode is available, the traces in $F_i^{(1)}$ are valued together using the reflection-weighted backfilling rule in Eq. 2. The same terminal feedback signal is shared within the episode, while each step receives a different value according to its reflection weight and temporal position. This is why MSCE does not treat each tool call as an independent training instance: doing so would make the feedback signal too sparse and unstable.

Role in L2 induction. Episodes also provide the independence criterion for policy induction. A single high-value trace may be associated with an existing L2 policy or placed into the candidate pool, but a new L2 policy is induced only when a candidate signature accumulates evidence from at least $n_{\min}$ distinct episodes. This prevents multiple traces from the same execution trajectory from being counted as independent support. In other words, the threshold for L2 induction is cross-episode support, whereas the value threshold $V(f) \geq v_{\min}$ is applied to individual L1 traces.

Role in retrieval. Although retrieval can return individual L1 traces, MSCE may also aggregate retrieved traces by episode. This episode-level rollup provides a compact reference trajectory when the current context resembles a previous execution segment, while still preserving the underlying trace-level evidence links.

B.3 Reward Quantification from Textual Feedback

When no numerical environment reward is available, MSCE converts textual user feedback into a normalized scalar $R_i \in [-1, 1]$. Given user feedback h_i and a compact task summary, an LLM evaluator scores the execution trajectory along three axes:

- **Goal achievement** $g_i \in [-1, 1]$: whether the agent addressed the user’s actual request, with later turns weighted more strongly in multi-turn tasks.
- **Process quality** $p_i \in [-1, 1]$: whether the execution was efficient, safe, and avoided unnecessary tool use or exploration.
- **User satisfaction** $u_i \in [-1, 1]$: whether the user’s feedback expresses acceptance, neutrality, correction, or rejection.

The terminal reward is computed as

$$R_i = \text{clip}(0.45g_i + 0.30p_i + 0.25u_i, -1, 1).$$

If the LLM evaluator is unavailable, MSCE falls back to a heuristic polarity-based estimate from the explicit feedback channel.

B.4 Policy Gain for L2

Sets. L2 policy gain is recomputed whenever reward backfilling finishes for an episode. For each touched policy, we form a finite trace pool $\mathcal{T}$ consisting of traces from the triggering episode and any additional traces that served as cross-episode induction evidence for that policy. The with set is $S_{\text{with}} = \{f \in \mathcal{T} \mid f \text{ is linked to the policy in this update}\}$ including traces matched by association and traces in the inducing bucket. The without set is $S_{\text{without}} = \mathcal{T} \setminus S_{\text{with}}$. Earlier in the pipeline, only traces with $V(f) \geq v_{\min}$ enter association or the candidate pool; gain computation itself uses the stored per-step values $V(f)$.

With-side aggregate. For $|S_{\text{with}}| \geq 3$, MSCE uses a value-weighted mean:

$$\mu_w(S_{\text{with}}) = \sum_{f \in S_{\text{with}}} w_f V(f), \quad (3)$$

where

$$w_f = \frac{\exp\left(\frac{(V(f) - \max_{g \in S_{\text{with}}} V(g))}{\tau_V}\right)}{\sum_{g \in S_{\text{with}}} \exp\left(\frac{(V(g) - \max_{h \in S_{\text{with}}} V(h))}{\tau_V}\right)}. \quad (4)$$

We use $\tau_V = 0.5$. For $|S_{\text{with}}| < 3$, the with-side aggregate falls back to the arithmetic mean:

$$\bar{V}_{\text{with}} = \mu(S_{\text{with}}) = \frac{1}{|S_{\text{with}}|} \sum_{f \in S_{\text{with}}} V(f).$$

Otherwise, $\bar{V}_{\text{with}} = \mu_w(S_{\text{with}})$.

Without-side aggregate. The without side uses an arithmetic mean, not a softmax-weighted mean:

$$\mu(S_{\text{without}}) = \frac{1}{|S_{\text{without}}|} \sum_{f \in S_{\text{without}}} V(f),$$

with $\mu(S_{\text{without}}) = 0$ when $S_{\text{without}} = \emptyset$. To avoid making the gain undefined or uninformative when few comparable failures are observed, MSCE shrinks this mean toward a conservative baseline $b = 0.5$ with pseudocount $N_0 = 5$:

$$\bar{V}_{\text{blend}}(S_{\text{without}}) = \frac{|S_{\text{without}}| \mu(S_{\text{without}}) + N_0 b}{|S_{\text{without}}| + N_0}. \quad (5)$$

Thus, if no without evidence is available, $\bar{V}_{\text{blend}} = b$.

Gain. The final policy gain is

$$G(f^{(2)}) = \bar{V}_{\text{with}} - \bar{V}_{\text{blend}}(S_{\text{without}}).$$

This gain is a heuristic utility estimate, not a causal effect estimate. Softmax weights are used only in gain recomputation; they do not duplicate or reweight examples inside the L2 induction prompt.

B.5 L2 Association, Induction, and Revision

For each valued L1 trace, MSCE first attempts association with an existing L2 policy. The matching score combines semantic similarity with structured trigger overlap. Incompatible domain tags or conflicting error signatures prevent association even when embedding similarity is high. Associated traces update the policy’s support set and gain estimate.

If no policy matches, the trace is placed into a candidate pool. A candidate pool becomes eligible for induction when it contains evidence from at least $n_{\min}$ distinct episodes. This independence requirement prevents a single long trajectory from producing an over-specific policy. When new evidence contradicts an active policy, MSCE does not directly overwrite its procedure. Instead, the policy gain is recomputed; sustained negative gain retires the policy, while substantial procedural drift triggers re-induction and possible skill rebuilding.

Algorithm 1 Environmental cognition abstraction

Require: active policies $\mathcal{P}$; similarity thresholds $\theta_{\text{sim}}, \theta_{\text{merge}}$; minimum cohort size m

```
1: Partition  $\mathcal{P}$  into domain buckets  $B$ 
2: for each  $b \in B$  with  $|b| \geq m$  do
3:    $C \leftarrow \text{ADMIT}(b, \theta_{\text{sim}})$  // strict centroid or tag fallback
4:    $\mathcal{T} \leftarrow$  one recent trace per policy in  $C$ 
5:    $d \leftarrow \text{LLM}(\Pi_{\text{env}}, C, \mathcal{T})$ 
6:   if  $d$  invalid or  $\text{conf}(d) = 0$  then
7:     continue
8:   end if
9:    $w^* \leftarrow \arg \max_w \cos(\vec{C}, \vec{w})$ 
10:  if  $\cos(\vec{C}, w^*) \geq \theta_{\text{merge}}$  then
11:    merge  $d$  into  $w^*$ 
12:  else
13:    insert  $d$  as a new environmental cognition
14:  end if
15: end for
```

B.6 L3 Abstraction and Confidence

Given active L2 policies, we bucket by a deterministic domain key, admit a cohort via centroid cosine similarity (with a tag-aligned fallback when embeddings diverge), and call the abstraction prompt Π_{env} (Appendix E) on the cohort plus one capped evidence trace per policy. The prompt is constrained to declarative environment knowledge and forbidden from prescribing actions. Validated drafts merge into the nearest existing environmental cognition by embedding similarity or instantiate a new environmental cognition. Algorithm 1 summarizes the procedure.

The confidence c of an L3 cognition is increased when newly induced policies support its existing facts and decreased when later evidence contradicts them. Low-confidence cognitions are not deleted immediately, since they may still serve as negative or historical evidence; instead, they are excluded from default retrieval until additional support appears.

B.7 Online Update Operator $\mathcal{U}$

Algorithm 2 formalizes the event-driven chain. Stages are idempotent per episode identifier; LLM stages are skipped when no provider is configured, while L1 persistence still proceeds.

B.8 Skill Lifecycle

Skills pass through three states: probationary, active, and archived. A newly crystallized skill is probationary. It becomes active when its reliability η exceeds a threshold after sufficient invocations. It is archived when repeated failures, explicit user rejection, or negative source-policy gain indicate that it should no longer be retrieved by default.

Reliability is estimated by a smoothed success rate:

$$\eta = \frac{n_{\text{pass}} + 1}{n_{\text{trial}} + 2}.$$

In addition, explicit positive or negative user feedback can adjust η , and boundary violations shrink the applicability boundary $\mathcal{B}$. When the source L2 policy is substantially revised, the corresponding skill is rebuilt from a fresh evidence set rather than patched in place. The detailed lifecycle operations for crystallized skills are provided in Table 4.

B.9 Hierarchical Retrieval

At inference time, MSCE retrieves memory in three tiers.

Algorithm 2 Online update operator $\mathcal{U}$

```
1: Upon episode finalization:
2:   Extract L1 steps
3:   Truncate the text
4:   Extract or synthesize self-reflection  $\rho$ 
5:   Score  $\alpha$  via  $\Pi_{\text{reflexion\_score}}$ 
6:   Embed summaries;
7:   Write traces with  $V = \text{PENDING}$ .
8: Upon receipt of  $R_i$  (feedback or timeout):
9:   Build an episode summary
10:   $R_i \leftarrow$  environment reward or  $\text{LLM}(\Pi_{\text{reward}}, \cdot)$ 
11:  Backfill  $V$ 
12:  Decay retrieval priorities
13: Upon reward update:
14: for traces with  $V \geq v_{\min}$  do
15:   Associate or pool
16:   Induce if quorum met
17:   Recompute ( $G$ , support, status)
18: end for
19: Upon active L2 change:
20:   Abstract L3 clusters
21:   Evaluate skill eligibility
22:   Crystallize  $\rightarrow$  verify  $\rightarrow$  probationary deploy
23: Upon failure burst or user correction:
24:   Synthesize contrastive guidance
25:   Append to  $\mathcal{D}$ 
26:   Stash repair packet for the next turn
27: On failure:
28:   Malformed JSON  $\Rightarrow$  skip stage
29:   Empty evidence  $\Rightarrow$  no new skill
```

Skill retrieval. The router first matches the current context against active skill triggers. Matched skills are ranked by trigger relevance, reliability η , and source-policy gain. The selected skill contributes its procedure, preconditions, verification rule, boundary, evidence anchors, and decision guidance to the agent context.

Trace and episode retrieval. If no skill applies, or if a skill fails, MSCE retrieves L1 evidence. Trace retrieval combines exact cues such as error signatures with semantic similarity over compact state summaries. High-value traces are prioritized, while low-value traces are preserved for counter-evidence and decision repair.

Environmental cognition retrieval. When the agent expresses structural uncertainty, such as not knowing where files, tests, configurations, or constraints are located, MSCE retrieves relevant L3 environmental cognition. The retrieved L3 cognition provides environmental priors but does not directly prescribe actions.

B.10 Decision Guidance and Repair

MSCE converts repeated contrastive evidence into decision guidance $\mathcal{D}$. When two action patterns occur in similar contexts but lead to substantially different values, and the lower-valued pattern is supported by failures or user corrections, MSCE creates a guidance item:

$$d = (c, a^+, a^-, e, \xi),$$

where c is the context, a^+ is the preferred action, a^- is the action to avoid, e contains evidence links, and ξ is guidance reliability. Guidance is injected only when its context matches the current skill or policy trigger. Later feedback can increase, decrease, or retire ξ .

Table 4 Lifecycle operations for crystallized skills.

Event	Operation	Effect
successful invocation	reinforce	increase reliability
execution failure	repair	revise procedure or verification
user rejection	shrink	narrow applicability boundary
new counter-evidence	revise	add guidance or anti-patterns
source policy rewritten	rebuild	re-crystallize from evidence
long inactivity or low reliability	archive	remove from default retrieval

Table 5 L1 traces from two distinct tasks demonstrating cross-task L2 policy induction. Task-level feedback ($R_A = 0.8$, $R_B = 0.9$) is backpropagated using reflection weights (α) to compute step values (V).

Task	Trace	Environment	Action	Observation	Reflection (ρ)	α	V
A	$f_{1,1}^{(1)}$	Alpine Docker	<code>pip install lxml</code>	Error: <code>xmlsec1 not found</code>	“Alpine lacks C library <code>xmlsec1</code> ”	0.7	0.776
	$f_{1,2}^{(1)}$	Alpine Docker	<code>apk add... && pip...</code>	Success	“Needs system lib before compile”	0.5	0.80
B	$f_{2,1}^{(1)}$	Debian Docker	<code>pip install psycopg2</code>	Error: <code>pg_config not found</code>	“Missing PostgreSQL dev library”	0.6	0.864
	$f_{2,2}^{(1)}$	Debian Docker	<code>apt-get... && pip...</code>	Success	“Strategy effective”	0.4	0.90

B.11 Storage and Privacy Controls

Because L1 traces may contain user text, tool outputs, file paths, or environment-specific information, MSCE applies bounded persistence. Long fields are truncated, repeated adjacent steps are deduplicated, and sensitive strings are removed through rule-based redaction before storage. Higher-level memories store evidence identifiers rather than raw unbounded observations. These controls reduce memory bloat and limit propagation of sensitive content, although they do not eliminate all privacy risks.

C Example: Environment Dependencies

To illustrate the MSCE workflow more clearly, consider an agent resolving environment dependencies across two distinct tasks shown in Table 5.

In **Task A** (“Install `lxml` in Alpine”), the agent’s initial `pip install` fails. The agent reflects on the missing C-library, installs it via the package manager, and succeeds. The user’s positive feedback $R_A = 0.8$ is backpropagated using the reflection weights α , assigning high trace values V to these exploratory steps. For instance, assuming a discount factor $\gamma = 0.9$, the value of the key discovery step $f_{1,1}^{(1)}$ is computed via reflection-weighted backfilling as $V = 0.7 \times 0.8 + (1 - 0.7) \times 0.9 \times 0.8 = 0.776$. Since no existing L2 policy matches, these L1 traces enter the candidate pool.

Later, in **Task B** (“Deploy Django App”), a similar failure occurs with `psycopg2` on a Debian-based container. The user confirms the resolution, yielding another set of high-value traces.

Upon the arrival of Task B’s traces, the online update operator $\mathcal{U}$ detects a cross-task overlap (container environment, `pip` compilation failure, missing system libraries) between $f_{1,1}^{(1)}$ and $f_{2,1}^{(1)}$. This triggers the induction of an **L2 policy**:

- **Trigger:** `pip install` fails due to missing system libraries in a container.
- **Procedure:** Parse the missing component $\rightarrow$ identify the OS package manager (e.g., `apk` or `apt-get`) $\rightarrow$ install the corresponding `-dev` library $\rightarrow$ retry `pip install`.
- **Boundary:** Containers only; excludes native systems with pre-installed libraries.

As this L2 policy demonstrates stable positive gain, MSCE abstracts an **L3 environmental cognition** (e.g., the three-layer dependency structure of Python C-extensions) and crystallizes the policy into a **skill**, avoiding redundant trial-and-error in future tasks.

D Experimental Details

D.1 Experimental Settings

Evaluation Protocol On EvoAgentBench, for task i , the per-task success rate is $\text{Pass@1}_i = \frac{1}{n_i} \sum_{j=1}^{n_i} \mathbb{I}[r_{i,j} > \tau]$. The benchmark-level Pass@1 is the unweighted mean over tasks, $\text{Pass@1} = \frac{1}{N} \sum_{i=1}^N \text{Pass@1}_i$, with standard error computed across tasks. Regarding how reward r is assigned, we follow the EvoAgentBench settings. Information Retrieval uses LLM-as-a-judge with exact-match fallback ($r \in \{0, 1\}$). Reasoning uses normalized exact matching or optional LLM-based mathematical equivalence ($r \in \{0, 1\}$). Software Engineering follows the official SWE-bench **resolved** criterion ($r \in \{0, 1\}$). Code Implementation requires passing all hidden test cases ($r \in \{0, 1\}$). Knowledge Work uses LLM rubric scoring with a ClawWork-compatible cliff: scores below 0.6 are mapped to failure, and success requires $r > 0.6$.

Hyperparameters Table 6 lists the principal hyperparameters used in our implementation. These values are not intrinsic to MSCE and can be tuned for different environments.

Table 6 Principal MSCE hyperparameters.

Symbol	Meaning	Value
<i>Trace valuation and reward</i>		
γ	value backfilling discount	0.9
w_g	reward weights for goal	0.45
w_p	reward weights for process	0.30
w_u	reward weights for satisfaction	0.25
<i>L2 policy induction and gain</i>		
$n_{\min}$	distinct episodes for L2 induction	2
$v_{\min}$	minimum value for L2 association	0.1
τ_V	temperature for value weighting in policy gain	0.5
N_0	pseudo-count in policy gain	5
b	shrinkage anchor in policy gain	0.5
θ_G	gain threshold for skill promotion	0
<i>L3 environmental cognition abstraction</i>		
m	minimum policy cohort size for L3 abstraction	2
θ_{sim}	cohort admission similarity threshold	0.62
θ_{edge}	related-topic Jaccard edge threshold	0.34
<i>Skill crystallization and lifecycle</i>		
K	maximum evidence traces per skill	6
k	retrieved skills per task	3
n_{prob}	minimum probationary invocations before activation	1
$\theta_{\eta}^{\text{active}}$	reliability threshold for active retrieval	0.6
$\theta_{\eta}^{\text{archive}}$	reliability threshold for archiving	0.2
δ	explicit-feedback update step for η	0.1
<i>Evaluation</i>		
τ_{eval}	success threshold for Pass@1 unless KW	0; 0.6 for KW

D.2 Effect of Simulated Human Feedback

MSCE is designed to use not only numerical environment rewards but also textual user feedback. To examine whether textual episode-level feedback provides useful learning signals, we conduct an additional controlled study on EvoAgentBench with LLM-simulated human feedback. This experiment also provides an indirect validation of the reward quantification mechanism in Appendix B.3, since the simulated feedback is converted into scalar terminal feedback by the same quantification module used in MSCE.

Table 7 Effect of LLM-simulated human feedback on EvoAgentBench. C0 denotes MSCE without simulated feedback, and C1 denotes MSCE with simulated feedback injected during training. Cost is measured in chars for Math and turns for other domains, so cost comparisons should be made within each domain.

Domain	Pass@1 $\uparrow$			Cost $\downarrow$		
	C0	C1	Δ	C0	C1	Δ
Code	61.54	64.10	+2.56	1.97	1.74	-0.23
SE	53.85	61.54	+7.69	40.80	37.71	-3.09
KW	53.45	67.24	+13.79	14.97	16.90	+1.93
Math	47.00	47.00	+0.00	1140.9	1150.0	+9.1
IR	26.15	27.69	+1.54	8.74	8.40	-0.34

Setup. We compare two configurations. **C0** denotes the default MSCE setting without injected simulated human feedback. **C1** augments episode-level feedback with simulated human feedback generated by an LLM mentor (Opus 4.7 Max). For failed or blocked training trajectories, the mentor observes the task prompt, agent trajectory, failure or blocker signal, verifier result, and existing memory/reflection content, and produces a concise root-cause-aligned feedback message. The feedback is written in the same style and format as existing memory or reflection content, so that it does not introduce a new memory schema or additional test-time hints. This feedback is injected only during the online evolving phase and is then quantified into terminal feedback R_i using the reward quantification mechanism in Appendix B.3. At evaluation time, no simulated feedback is generated; the agent uses only the memory and skills accumulated during training.

Feedback generation prompt. The LLM mentor is instructed to behave like an experienced human coach rather than an oracle. It must not provide final answers, complete code, or task-specific shortcuts. Instead, it identifies what the agent was trying to do, where the reasoning or action deviated from the correct path, why the deviation caused failure or blockage, what general principle should be remembered, and which concrete evidence or check should be prioritized in similar future tasks. This design encourages feedback that is reusable as training memory rather than direct supervision for a particular evaluation instance.

Results. Table 7 compares **C0** and **C1**. Simulated feedback improves Pass@1 on four of the five domains, with the largest gains on KW and SE, where **C1** improves Pass@1 by +13.79 and +7.69 points, respectively. Code and IR also improve by +2.56 and +1.54 points, while Math remains unchanged, suggesting that its bottleneck may lie more in exact symbolic correctness than in procedural improvement driven by richer feedback.

The cost trend is mixed: **C1** reduces cost on Code, IR and SE but increases cost on KW, and Math. This suggests that richer feedback can reduce redundant exploration in some tool-use tasks, while encouraging more verification or elaboration in others. Overall, these results suggest that episode-level textual feedback can provide useful additional supervision for MSCE’s memory–skill evolution, and that the proposed reward quantification mechanism can convert such feedback into effective update signals.

E Prompted Operators

MSCE uses LLMs for five evidence-constrained operators: reflection scoring, reward quantification, L2 policy induction, and L3 environmental cognition abstraction, and skill drafting. All prompted operators are required to return structured fields and evidence identifiers. Outputs that lack required fields, refer to unsupported evidence, or introduce unsupported tools are discarded rather than inserted into memory.

E.1 Reflection Scoring

Given a step context $(s_{i,t}, a_{i,t}, o_{i,t}, \rho_{i,t})$, the reflection scorer estimates $\alpha_{i,t} \in [0, 1]$. The rubric considers four aspects: faithfulness to the observed step, causal insight, transferability to future tasks, and concreteness. Empty or tautological reflections receive $\alpha_{i,t} = 0$. If reflection scoring is disabled, non-empty reflections

receive a neutral value $\alpha_{i,t} = 0.5$; malformed judge outputs fall back to the existing reflection metadata rather than updating the trace. In batched mode, all steps in a short task are scored jointly so that the judge can compare their relative contribution. The specific prompt ($\Pi_{\text{reflexion_score}}$) is provided as follows:

```
You are a strict reviewer of agent self-reflections.

You see the FULL context of one agent step:
- STATE      what the agent saw before acting (user prompt, prior observation)
- THINKING    the LLM's own native chain-of-thought for this step, if any (Claude extended-thinking, pi-ai ThinkingContent).
               Empty when the model didn't emit thinking this turn.
- ACTION      what the agent produced (assistant text output)
- TOOL_CALLS  tools the agent invoked this step, with inputs and outputs (or errors). Tool usage + outcomes are part of the
               action chain and carry their own signal about what the agent did.
- OUTCOME     the final observable result of the step (last tool outcome or "(assistant-only step)" for pure text turns)
- REFLECTION  the text being graded: the agent's first-person explanation of WHY it acted this way and WHAT it learned.

Score the REFLECTION on four axes, combined into ONE number [0, 1]:

    1. faithfulness does the reflection match what ACTUALLY happened across THINKING + ACTION + TOOL_CALLS + OUTCOME?
    2. causal insight does it identify why the action / tool choice worked or failed? Bonus when it connects the model's visible
       THINKING to the resulting action.
    3. transferability does it surface a lesson useful on a similar future task?
    4. concreteness are the details specific (real command names, real error messages, real decisions) rather than generic
       platitudes like "I should do better"?

Rules:
- THINKING and TOOL_CALLS are first-class evidence for grading a reflection that ignores a visible thinking chain or
  misreports a tool call should score LOW on faithfulness.
- TOOL_CALLS that errored are strong signal: the reflection should name the error and what it implied. Missing that is a
  faithfulness penalty.
- An empty / purely-tautological reflection = 0, usable = false.
- 0.4 AND reflection non-tautological usable = true; else false.

Return JSON:
{
  "alpha": 0.0-1.0,
  "usable": true | false,
  "reason": "one-sentence justification"
}
```

E.2 Reward quantification.

When no numerical environment reward is available, textual user feedback is converted into R_i . The evaluator scores goal achievement, process quality, and user satisfaction. The input includes the original query q_i , the final output $\hat{y}_i$, a compact episode summary, and user feedback h_i . The specific reward quantification prompt (Π_{reward}) is as follows:

```
You are a strict grader of AI-agent task execution.

You receive:
- TASK_SUMMARY the FULL conversation arc for this task: * USER_ASKS_AND_AGENT_REPLIES lists every user turn paired with the
  agent's corresponding reply, in chronological order. One "task" frequently spans multiple user turns as the user refines
  / follows up / pivots topics within the same session. * MOST_RECENT_USER_ASK and MOST_RECENT_AGENT_REPLY call out the
  final exchange explicitly that is usually the truest signal of whether the agent is actually tracking where the user is
  now.
- FEEDBACK      the user's own messages AFTER the task attempt finished. May be short ("ok thanks"), explicit ("try again with
  X"), or structured ("resolved, but too slow"). Frequently empty.

Grade the agent on THREE INDEPENDENT AXES, each in [-1, 1]:

1. "goal_achievement" did the agent address what the user ACTUALLY asked?
   +1.0 every user ask across the exchange was addressed correctly.
   +0.3 the last ask was addressed well; earlier asks had minor gaps.
   0.0 unclear if the user's ask was met.
   -0.3 missed a significant portion of what was asked.
   -1.0 fundamentally wrong answer / caused damage.

   CRITICAL RULE do NOT anchor on the first user turn. A user who starts with "" and later pivots to "" is a user whose goal
   has EVOLVED; if the agent answered Beijing on the final turn when asked about Beijing, that is goal-achievement =
   POSITIVE, not negative. Judge each user ask on its own merits, weighted toward the most recent exchange (which is
   where the user actually is now).

2. "process_quality"
   +1.0 clean, minimal, correct reasoning across all turns.
   0.0 reasonable but not great.
```

```

-1.0 lots of thrashing, wrong tools, noisy output.

3. "user_satisfaction" (from FEEDBACK text tone + trailing user asks)
+1.0 thanks / happy / "" / accepts and closes out.
+0.3 moves on neutrally to next ask or new topic.
0.0 no emotional signal either way.
-0.3 asks for correction ("no, do X instead" / "").
-1.0 hard-stops, expresses frustration.

Rules:
- If FEEDBACK is empty, infer satisfaction CONSERVATIVELY from the last exchange's tone. A follow-up question is usually 0 (
  neutral continuation), NOT negative. Never invent anger.
- Base scores ONLY on what TASK_SUMMARY actually describes do not assume facts not shown.
- You are grading the HOST AGENT described in HOST_AGENT_CONTEXT, not yourself. Do NOT use your own model identity, provider,
  policies, or capabilities to decide whether the host agent answered identity/model questions correctly. If hostModel/
  hostProvider are provided, treat them as the authoritative runtime context unless the conversation itself contains a
  correction.
- Produce one short justification.

Return JSON, EXACTLY this shape (no extra keys, no commentary):
{
  "goal_achievement": number in [-1, 1],
  "process_quality": number in [-1, 1],
  "user_satisfaction": number in [-1, 1],
  "label": "success" | "partial" | "failure" | "unknown",
  "reason": "one-sentence justification"
}

```

E.3 L2 induction.

The L2 induction prompt receives evidence traces selected from distinct episodes. Each evidence item contains a compact state, action, outcome, value V , reflection weight α , and evidence id. The model is asked to produce a trigger, procedure, verification rule, applicability boundary, and supporting evidence ids. The prompt (Π_{policy}) is explicitly restricted to procedural knowledge: declarative environment facts are deferred to L3 abstraction as follows:

```

You induce reusable procedural policies from agent experience.

A policy is a "how-to": "when you see condition X in the agent's state, do action Y, verify with Z, watch out for caveat W."
It is NOT a description of the environment.

Input TRACES: a list of { state_summary, action, outcome, utility } records that all share a similar state signature.

Produce ONE policy describing the action pattern. The policy must:
- Name a TRIGGER recognizable from the agent's STATE a condition the agent can detect at the moment of decision (an error code,
  a missing file, a request shape). NOT a fact about the environment in general.
- Prescribe an ACTION template a parameterized step or short step sequence. Templates over single exact commands. NOT a single
  example.
- Note at least one CAVEAT or failure mode observed in the traces a step-level pitfall, NOT a generic environment taboo.
- Generalize across the input traces, not restate one of them.

Boundaries what NOT to write

This output is a procedural policy, not environmental cognition. Environmental cognition lives in a separate layer (L3)
generated by a different prompt. Cross-contamination on either side dilutes both.

Do NOT write any of these they belong to L3 (environmental cognition), not here:
- Topology facts: "Alpine containers ship musl libc" "Python deps form a 3-layer stack" "src/components/ holds React
  components"
- Environment behavioural rules (in pure declarative form): "binary wheels are incompatible with musl" "the service reads
  config only at startup"
- Environment taboos detached from a specific action choice: "this directory is read-only" "production tables shouldn't be
  DROPPed lightly"

If a trace tells you the environment looks a certain way, FOLD that fact INTO the trigger or caveat as a state-level CONDITION
the agent can check, not as a standalone description. Example:

Wrong (drifts into env-fact):
  trigger: "Alpine ships musl libc"
  caveats: ["Python deps have a 3-layer stack"]

Right (states it as actionable conditions):
  trigger: "container is Alpine AND pip install fails with '<lib> not found' or 'header not found'"
  caveats: ["if first apk add still fails, also check musl-vs-glibc wheel compatibility before retrying"]

Same fact, two framings

```

If the underlying truth is "Alpine containers don't ship system dev libs by default":

Express here (procedural): "When pip install fails inside an Alpine container with a missing system library, run apk add <pkg>-dev then retry pip."

Do NOT express here (declarative that's L3's job): "Alpine container images ship only the pure-Python tier of the Python dependency stack."

Output

Return JSON:

```
{
  "title": "short imperative title",
  "trigger": "state-level condition the agent can detect",
  "action": "templated step or step sequence",
  "rationale": "why this action works ON THESE TRACES (not why the
               environment behaves this way)",
  "caveats": ["step-level pitfall string", ...],
  "confidence": number in [0, 1],
  "support_trace_ids": ["tr...", ...]
}
```

E.4 L3 abstraction.

The L3 abstraction prompt receives a cohort of active L2 policies and representative evidence. It extracts declarative environment knowledge in three fields: entities or structures $\mathcal{E}$, action-response regularities $\mathcal{I}$, and constraints $\mathcal{C}$. The prompt (Π_{env}) forbids imperative procedural instructions, so that L3 remains an environmental cognition rather than a skill as follows:

You abstract environmental cognition from cross-task policy evidence.

An environmental cognition model is **declarative** knowledge about how the environment IS: its topology, its causal/behavioural regularities, its taboos. It is **NOT** a recipe for what to do that lives in the L2 procedural layer, generated by a separate prompt. Cross-contamination on either side dilutes both.

Input POLICIES: a list of L2 policies (with trigger / procedure / verification / boundary / support / gain), plus a short sample of the L1 traces that minted each. Every policy shares a compatible domain (matched by primary tag / tool).

Produce ONE environmental cognition model describing the **environment** those policies operate in. It must answer:

- Environment topology () what lives where, what is the shape of this environment? Pure facts of existence and structure.
GOOD: "Alpine containers ship musl libc, no glibc"
"Node project repos group source under src/"
"macOS bundles BSD sed; Linux distros bundle GNU sed"
BAD (drifts into procedure):
"use apk add to install system libs"
"prefer Python scripts over sed on macOS"
- Inference rules () how does the environment causally respond to common stimuli? Phrase as causeeffect, NOT as guidance.
GOOD: "loading a glibc-linked binary wheel inside Alpine raises a dynamic-link error" "editing config.yaml does not propagate until the process restarts (no in-process watcher)"
BAD (drifts into procedure): "if pip install fails, install dev libs and retry" that's an action plan, belongs to L2 "always restart the service after editing config" that's a recommendation, belongs to L2
- Constraints (C) what facts of the environment make some actions unsafe or invalid? State the FACT, not the avoidance behavior.
GOOD: "node_modules/ is rewritten by npm install; manual edits are lost on the next sync" "production database tables hold customer data; destructive DDL is irreversible"
BAD (drifts into procedure): "don't edit node_modules/ directly" that's a behavioural rule, belongs to L2 / decision repair
"don't run DROP TABLE in production" same phrase the underlying environment fact instead

Do NOT, under any section:

- Use imperative or recommendation verbs (do / don't / should / use / prefer / avoid / try / install / run). The environmental cognition never tells the agent what to do.
- Restate a single trace the cognition model must generalise across policies.
- Include advice tied to a single user or session.

Same fact, two framings

If the underlying truth is "Alpine containers don't ship system dev libs by default":

Express here (declarative):
inference: "Python C-extension packages fail to compile in Alpine"

```

        containers when the matching system header / library
        package is not pre-installed in the image."

Do NOT express here (procedural that's L2's job):
    "When pip fails in Alpine, apk add <pkg>-dev and retry pip."

Output

Return JSON:
{
  "title": "short noun phrase, e.g. 'Alpine python dependency model'",
  "domain_tags": ["tag1", "tag2"], // 1-4 short, lowercase, no spaces
  "environment": [
    { "label": "...", "description": "...", "evidenceIds": ["po...", "tr..."] }
  ],
  "inference": [ { "label": "...", "description": "...", "evidenceIds": [] } ],
  "constraints": [ { "label": "...", "description": "...", "evidenceIds": [] } ],
  "body": "rendered markdown summary of the three sections",
  "confidence": number in [0, 1],
  "supersedes_cognition_ids": [] // optional: prior cognition this refines
}

```

E.5 Skill drafting.

The skill crystallization prompt receives the source L2 policy, ranked supporting evidence, optional counter-examples, a whitelist of observed tools, decision-guidance seeds, and the existing skill naming space. It returns a callable procedure with preconditions, ordered steps, examples, and decision guidance. The generated skill is accepted only if its tools and claims are supported by the evidence set. The specific prompt (Π_{skill}) is as follows:

```

You crystallize a skill an agent should be able to call.

Input:
- POLICY: the L2 policy being promoted (trigger / action / rationale / caveats).
- EVIDENCE: 3..10 successful traces that support the policy.
- EVIDENCE_TOOLS: the exhaustive list of tool/command names that actually appeared in the evidence traces' tool calls. This is
  the ground-truth
  whitelist your 'tools' output MUST be a subset of this list.
- COUNTER_EXAMPLES (optional): traces with V < 0 from the same context
  failures the policy is meant to prevent.
- REPAIR_HINTS (optional): a JSON block { preference: [...], antiPattern: [...] } attached to the policy by the decision-
  repair pipeline. These are concrete "prefer / avoid" lines synthesised from earlier failures + user feedback; treat them
  as authoritative seeds for 'decision_guidance' below.
- NAMING_SPACE: a list of existing skill names to avoid colliding with.

Return JSON:
{
  "name": "snake_case_identifier, 32 chars, unique vs NAMING_SPACE",
  "display_title": "human title in user's language",
  "summary": "2-3 sentence description of what the skill does and when to use it",
  "parameters": [
    { "name": "...", "type": "string|number|boolean|enum", "required": true|false,
      "description": "...", "enum": [...] }
  ],
  "preconditions": ["bullet", ...],
  "steps": [
    { "title": "short", "body": "markdown-friendly paragraph describing the step" }
  ],
  "examples": [
    { "input": "...", "expected": "..." }
  ],
  "tools": ["tool_or_command_name", ...],
  "decision_guidance": {
    "preference": ["Prefer: ", ...], // concrete actions to favour, 5
    "anti_pattern": ["Avoid: ", ...] // concrete actions to avoid, 5
  },
  "tags": ["optional string", ...]
}

Rules:
- 'tools' MUST only contain names from EVIDENCE_TOOLS. Never invent tool names that are not in the whitelist. Include every
  tool the skill's procedure actually invokes omit tools not referenced in your steps.
- Keep "steps" short (2-6 items).
- 'summary' must be self-contained so the agent can decide whether to call this skill without reading the full SKILL.md.
- For 'decision_guidance':

```

- If REPAIR_HINTS is non-empty, fold each line in verbatim (or lightly normalised) they are already grounded in evidence and user feedback.
- You MAY add 12 extra entries derived from contrasting EVIDENCE (high-V) vs COUNTER_EXAMPLES (low-V), if they materially clarify the decision. Don't invent guidance unsupported by the inputs.
- Each entry should be one short, actionable sentence (200 chars).
- Empty arrays are fine when there's nothing to say never fabricate.